The Design of *HAL9000* Operating System

Students' Name

October 23, 2026

Abstract

The abstract should contain a very short description of the results presented in this report.

Take into account that a design document is a high-level, logical (abstract) description of the solution you propose to the problems and requirements you dealt with. Though, it should be detailed enough such that somebody who already knows and understands the requirements could implement (i.e. write the code) the design without having to take any significant additional design decisions. Even further, if such an implementer would happen to be an experienced one, he or she should be able to use the design document to implement the solution in just few hours (not necessarily including the debugging).

Important notes. Take care to remove from the given design document template all the text that was given to you as a guideline and provide your own document with only your own original text. Also, do not simply write anything in any section just to have some text there, but only write text that makes sense in the given context. We will not count the number of pages or words of your document, but will only evaluate the meaning and completeness of the written words.

Chapter 1

General Presentation

1.1 Working Team

Specify the working team members' names and their responsibility.

1. Firstname1 Familyname1
   (a) Threads: dealt with AAA
   (b) Userprog: dealt with BBB
   (c) VM: dealt with CCC
2. Firstname2 Familyname2
   (a) Threads: dealt with AAA
   (b) Userprog: dealt with BBB
   (c) VM: dealt with CCC
3. Firstname3 Familyname3
   (a) Threads: dealt with AAA
   (b) Userprog: dealt with BBB
   (c) VM: dealt with CCC

Chapter 2

Required Template for the Design of Each Project's Module

Notes

The final design document delivered for each module must not exceed 2 pages per student task and 7 pages in total.

The examples given in each section/subsection are not complete/exhaustive!

2.1 Module Overview and Requirements

This section will contain the requirements for each module. Read it carefully and don't change it or remove it.

2.2 Design Details

2.2.1 Design Description

Here you should describe the design of your solution. This should be the main and most substantial part of the document.

The solution should be described in terms of the main data structures and the functions or interfaces you will introduce or modify.

- the key invariants (properties that must always be true) of your code and how they maintained;
- how shared state is protected and concurrent operations are coordinated;

- how and where resources are allocated, used, and released;
- how errors and edge cases are handled.

Recommendations:

- Use pseudocode to describe algorithms
- Use diagrams to illustrate the architecture your solution
- Don't list every minor helper function
- Don't include existing declarations without explanation
- Don't provide a line-by-line description of the implementation

Example

Timer We store a list of active timers, each with a list of waiting threads. The list of timers is checked periodically to see if any of them expired. When a timer expires, all waiting threads are released.
To block the threads waiting for a timer, we use executive events, which allow blocking threads with `ExEventWaitForSignal`. To unblock all waiting threads, we initialize the event as `ExEventTypeNotification` and call `ExEventSignal`.
The events are initialized and uninitialized together with their timer, with the observation that they should be signaled before the timer is destroyed.
Priority Scheduler. Fixed Priority Scheduler We change how threads are inserted into the waiting lists, so they are ordered descending by priority, using `InsertOrderedList`. To ensure that the highest priority threads are running at any given time, we will send an IPI (interprocessor interrupt) to invoke the scheduler on each CPU. Specifically, we need to do this when:

- a thread is inserted into the global ready list, when a new thread is created or when a thread is unblocked from a timer, mutex or event.
- the priority of threads changes, due to priority modifications or priority donation.

Priority Scheduler. Priority Donation Priority donation will be structured like a tree with two kinds of nodes:

- thread - each thread has a list of held mutexes
- mutex - each mutex has a list of waiting threads

The mutex nodes only play an "intermediary" role, passing the priority of the threads waiting for the respective mutex to the holder of the mutex. This donation can happen recursively, with a chain like $Thread_A \xrightarrow{Waits} Mutex_1 \xrightarrow{Donates} Thread_B \xrightarrow{Waits} Mutex_2 \xrightarrow{Donates} Thread_C$, which must be handled by "propagating" the priority down to the running thread (in this case $Thread_C$).
We also need to consider the cases when a thread holds multiple mutexes, choosing the highest priority of all donations, and when a mutex has multiple waiting threads, again, choosing the highest priority.

Task A

Student TODO: *Complete the text for Task A*

Task B

Student TODO: *Complete the text for Task B*

Task C

Student TODO: *Complete the text for Task C*

2.2.2 Interactions Between Components (Student Tasks)

Here you should describe each case where the code/logic for a given component (student task) depends on other components (student tasks). After you identify the interactions, we recommend defining an interface for each component, so the users of the given component (i.e. your teammates who implement the other tasks) don't need to know its internal details.

Given that the implementation is in C, an interface will be a header file containing function headers and type definitions. Before starting the actual implementation work (i.e. modifying `.c` files), each team member should create or extend one or more (existing) header (`.h`) files, that expose the functions and types that will be used by their teammates when implementing the other modules.

Example

Timer There is no direct interaction between the Timer and the Scheduler or Donation. There is an indirect interaction, because the timer makes use of executive events, which need to take into consideration thread priorities when releasing the waiting threads.
Priority Scheduler and Priority Donation The priority donation component depends on the scheduler, because it must be notified whenever thread priorities change.
The priority scheduler should expose a function to the priority donation component, so the priority donation can "notify" the scheduler that a priority changed during donation and it may need to reschedule threads.

Student TODO: *Complete the text as a team, defining a subsubsection for each interaction, or to note the lack of interactions*

2.2.3 Design Decisions, Motivations and Alternatives

Here you should explain the key design decisions you made and the motivations behind them. Discuss any alternative approaches you considered and why you rejected them.

Recommendations:

- discuss realistic alternatives, that you actually considered
- describe each alternative briefly, without going into significant details

Example

Timer The timers are checked in the system timer interrupt (`ExSystemTimerTick`), which is simple and already provided, but its main disadvantage is that it's not precise. An alternative that should have significantly better precision is to use ACPI or PIT timer, by programming the next interrupt to be exactly at the trigger time of the next timer. This requires both a significant amount of research to understand how these mechanisms and their implementation works, so we discarded this option.

Priority Scheduler. Fixed Priority Scheduler The data structure used to store waiting/blocked threads was implemented as a doubly-linked circular list, and uses `InsertOrderedList`, because this required the least amount of implementation work, and is easy to understand. The main alternative is using a more efficient data structure, for example, an array of doubly-link circular lists, with one list for each priority level. While this would be more efficient, it would also use more memory and would take more effort to implement.

Priority Scheduler. Priority Donation In the "donation tree", we always propagate the priority of the donor thread to the root, because it's easy to prove correct. We considered if this is really needed, and if we could stop early when we see a mutex or thread with a higher priority down the chain, but discarded the idea because we weren't sure that we covered all edge cases.

Task A

Student TODO: *Complete the text for Task A*

Task B

Student TODO: *Complete the text for Task B*

Task C

Student TODO: *Complete the text for Task C*

2.3 Implementation Considerations

Here you should describe the practical aspects that you needed to take into consideration during the implementation. This can include:

- main challenges and issues encountered during the implementation
- interesting bugs encountered, how you found the root cause, how you fixed them and how you could prevent similar bugs in the future
- test cases that required more effort to pass than you expected

Example

Timer We noticed that all timer tests already passed before starting the implementation, but they started to fail once we introduced partial modifications into the code.
Priority Scheduler. Fixed Priority Scheduler To force other CPUs to yield, we used `SmpIpiSendToAllIncludingSelf` and initially called `ThreadYield` inside the IPI, but we noticed that the system would deadlock after a while. We then understood that functions that invoke the scheduler can't be called inside an interrupt. So to fix the issue, we instead set the `YieldOnInterruptReturn` field of the current CPU.

Task A

Student TODO: *Complete the text for Task A*

Task B

Student TODO: *Complete the text for Task B*

Task C

Student TODO: *Complete the text for Task C*

Chapter 3

Design of Module *Threads*

Notes

The final design document delivered for each module must not exceed 2 pages per student and 7 pages in total.

3.1 Module Overview and Requirements

Timer

We must change the given executive timer functionality, such that to not use anymore the busy waiting technique in function `ExTimerWait()`, but a blocking mechanism, which suspends the waiting thread (i.e. takes the CPU from it) until the waited timer expires.

Priority Scheduler. Fixed Priority Scheduler

We must implement a priority-based thread scheduling policy (algorithm), which means the scheduler must consider threads' priorities when deciding which thread to be given an available CPU. The main scheduling rule is that when a thread must be chosen, the one with the higher priority must be the choice. The scheduler must be preemptive, which means it must always assure that threads with the highest priorities (considering multiple CPUs) are the ones run at any moment. This could suppose a currently running thread could be suspended, when a new higher-priority thread occurs.

Priority Scheduler. Priority Donation

We must implement (temporary) priority donation as a solution to the "priority inversion" problem. Priority inversion correspond to situations when a thread with a higher priority wait for a thread with a smaller priority (contrary to the priority-based rule, which requires the opposite). Such a situation occur when a thread with a smaller priority succeeds taking a lock, which will be later on required by a higher-priority thread. In order to avoid having the higher-priority thread waiting for smaller-priority threads (others than the lock holder, but having a higher priority than it), the higher thread donates its priority to the lock holder (smaller thread) until the lock will be released. In the meantime, no other "in-between" thread, could block the lock holder and consequently the waiting higher-priority thread.

3.2 Design Details

3.2.1 Design Description

Timer

Student TODO: *Complete the text for Timer*

Priority Scheduler. Fixed Priority Scheduler

Student TODO: *Complete the text for Priority Scheduler. Fixed Priority Scheduler*

Priority Scheduler. Priority Donation

Student TODO: *Complete the text for Priority Scheduler. Priority Donation*

3.2.2 Interactions Between Components (Student Tasks)

Student TODO: *Complete the text as a team, defining a subsubsection for each interaction, or to note the lack of interactions*

3.2.3 Design Decisions, Motivations and Alternatives

Timer

Student TODO: *Complete the text for Timer*

Priority Scheduler. Fixed Priority Scheduler

Student TODO: *Complete the text for Priority Scheduler. Fixed Priority Scheduler*

Priority Scheduler. Priority Donation

Student TODO: *Complete the text for Priority Scheduler. Priority Donation*

3.3 Implementation Considerations

Timer

Student TODO: *Complete the text for Timer*

Priority Scheduler. Fixed Priority Scheduler

Student TODO: *Complete the text for Priority Scheduler. Fixed Priority Scheduler*

Priority Scheduler. Priority Donation

Student TODO: *Complete the text for Priority Scheduler. Priority Donation*

3.4 Questions that you could be asked

This section must be removed. This is only to give you some hints for your design.

Some questions you have to answer (inspired from the original Pintos design templates), but these are not the only possible questions and we insist that your design should not be based exclusively to answering such questions:

1. timer

    - Briefly describe what happens in a call to *ExTimerWait()*, including the effects of the timer interrupt handler.
    - What steps are taken to minimize the amount of time spent in the timer interrupt handler?
    - How are race conditions avoided when multiple threads call *ExTimerWait()* simultaneously?

Table 3.1: MLFQ Tracing Example

timer ticks	recent_cpu			priority			thread to run
	A	B	C	A	B	C	
0							
4							
8							
12							
16							
20							
24							
28							
32							
36							

- How are race conditions avoided when a timer interrupt occurs during a call to *ExTimerWait()*?

2. priority scheduler

   - How do you ensure that the highest priority thread waiting for a mutex or executive event wakes up first?
   - Describe the sequence of events when a call to *MutexAcquire()* causes a priority donation. How is nested donation handled?
   - Describe the sequence of events when *MutexRelease()* is called on a lock that a higher-priority thread is waiting for.
   - Describe a potential race in *ThreadSetPriority()* and explain how your implementation avoids it. Can you use a lock to avoid this race?

3. advanced scheduler (MLFQ)

   - Suppose threads A, B, and C have nice values 0, 1, and 2. Each has a *recent_ cpu* value of 0. Fill in the Table B.1 (note: you can use `http://www.tablesgenerator.com/` to easily generate Latex tables) showing the scheduling decision and the priority and *recent_ cpu* values for each thread after each given number of timer ticks:
   - Did any ambiguities in the scheduler specification make values in the table uncertain? If so, what rule did you use to resolve them? Does this match the behavior of your scheduler?

- How is the way you divided the cost of scheduling between code inside and outside interrupt context likely to affect performance?

Chapter 4

Design of Module *Userprog*

Notes

The final design document delivered for each module must not exceed 2 pages per student and 7 pages in total.

4.1 Module Overview and Requirements

Remove the following given official requirements and describe in few words (your own words) what you are requested to do in this part of your project. Even if this is something that we all know, it could be a good opportunity for you to be sure you really understand this aspect.

The major requirements of the "Userprog" assignment, inspired from the original Pintos documentation, are the following:

- *System Calls for Process Management.* You have to implement the system calls *SyscallProcessExit()*, *SyscallProcessCreate()*, *SyscallProcessGetPid()*, *SyscallProcessWaitForTermination()* and *SyscallProcessCloseHandle()*.

- *System Calls for Thread Management.* You have to implement the system calls *SyscallThreadExit()*, *SyscallThreadCreate()*, *SyscallThreadGetTid()*, *SyscallThreadWaitForTermination()* and *SyscallThreadCloseHandle()*.

- *Program Argument Passing.* You have to change new process creation, such that the program arguments to be passed on its user-space stack.

- *System Calls for File System Access.* You have to implement system calls for opening existing files or creating new files (*SyscallFileCreate()*),

reading data from a file *SyscallFileRead()* and closing a file *SyscallFileClose()*).

Some additional (and optional) requirements of the "Userprog" assignment, specific to UTCN / CS OSD course could be:

- *IPC mechanisms.* You have to add in-kernel support for IPC mechanisms (pipes, shared memory) and the corresponding system calls (also including synchronization mechanisms) for user applications.
- *Dynamic Memory Allocation Support.* Add in kernel support for mapping new areas in the application's virtual address space and also support managing dynamically allocated memory and corresponding system calls.
- *Code Sharing Support.* Add support for sharing common code of different processes.

The way to allocate requirements on member teams.

- 3-members teams
  1. argument passing + validation of system call arguments (pointers)
  2. system calls for process management + file system access
  3. system calls for thread management

4.2 Design Details

4.2.1 Design Description

Program Argument Passing

Student TODO: *Complete the text for Program Argument Passing*

System Calls for Process and Thead Management

Student TODO: *Complete the text for System Calls for Process and Thead Management*

System Calls for Filesystem Access

Student TODO: *Complete the text for System Calls for Filesystem Access*

4.2.2 Interactions Between Components (Student Tasks)

Student TODO: *Complete the text as a team, defining a subsubsection for each interaction, or to note the lack of interactions*

4.2.3 Design Decisions, Motivations and Alternatives

Program Argument Passing

Student TODO: *Complete the text for Program Argument Passing*

System Calls for Process and Thead Management

Student TODO: *Complete the text for System Calls for Process and Thead Management*

System Calls for Filesystem Access

Student TODO: *Complete the text for System Calls for Filesystem Access*

4.3 Implementation Considerations

Program Argument Passing

Student TODO: *Complete the text for Program Argument Passing*

System Calls for Process and Thead Management

Student TODO: *Complete the text for System Calls for Process and Thead Management*

System Calls for Filesystem Access

Student TODO: *Complete the text for System Calls for Filesystem Access*

4.4 Questions that you could be asked

This section must be removed. This is only to give you some hints for your design.

Some questions you have to answer (inspired from the original Pintos design templates), but these are not the only possible questions and we insist that your design should not be based exclusively to answering such questions:

1. argument passing

   - Briefly describe how you implemented argument parsing. How do you arrange for the elements of argv[] to be in the right order? How do you avoid overflowing the stack page?
   - Why does *HAL9000* implement *strtok_s()* but not *strtok()*?

2. system calls

   - Describe how handles are associated with files, processes or threads. Are handles unique within the entire OS or just within a single process?
   - Describe your code for reading and writing user data from the kernel.
   - Suppose a system call causes a full page (4,096 bytes) of data to be copied from user space into the kernel. What is the least and the greatest possible number of inspections of the page table (e.g. calls to *_VmIsKernelAddress()*) that might result? What about for a system call that only copies 2 bytes of data? Is there room for improvement in these numbers, and how much?
   - Briefly describe your implementation of the "SyscallProcessWaitForTermination" system call and how it interacts with process termination.
   - Any access to user program memory at a user-specified address can fail due to a bad pointer value. Such accesses must cause the system call to fail. System calls are fraught with such accesses, e.g. a "SyscallFileRead" system call requires reading the function's four arguments from the user stack then writing an arbitrary amount of user memory, and any of these can fail at any point. This poses a design and error-handling problem: how do you best avoid obscuring the primary function of code in a morass of error-handling? Furthermore, when an error is detected, how do you ensure that all temporarily allocated resources (locks, buffers, etc.) are freed? In a few paragraphs, describe the strategy or strategies you adopted for managing these issues. Give an example.
   - Consider parent process P with child process C. How do you ensure proper synchronization and avoid race conditions when P calls SyscallProcessWaitForTermination(C) before C exits? After C exits? How do you ensure that all resources are freed in each

case? How about when P terminates without waiting, before C exits? After C exits? Are there any special cases?

Chapter 5

Design of Module *virtualmemory*

Notes

The final design document delivered for each module must not exceed 2 pages per student and 7 pages in total.

5.1 Module Overview and Requirements

Remove the following given official requirements and describe in few words (your own words) what you are requested to do in this part of your project. Even if this is something that we all know, it could be a good opportunity for you to be sure you really understand this aspect.

The requirements of the "Virtual Memory" assignment are the following:

- *Per Process Quotas.* You have to limit the number of open files a process can have and the number of physical frames it may use at one time.
- *Zero Pages.* You have to implement support for allocating virtual pages filled with zero bytes.
- *System calls.* You have to implement two new system calls for allocating and de-allocating virtual memory.
- *Stack Growth.* You have to add support for dynamically allocated stack.
- *Swapping.* You have to implement a page replacement algorithm (i.e. second chance) and swapping.
- *Shared Memory.* You have to add support for multiple processes to share the same memory region.

The way to allocate requirements on member teams.

- 3-members teams
    1. Per Process Quotas + Zero Pages
    2. System calls + Stack Growth
    3. Swapping

5.2 Design Details

5.2.1 Design Description

Per Process Quotas and Zero Pages

Student TODO: *Complete the text for Per Process Quotas and Zero Pages*

System Calls and Stack Growth

Student TODO: *Complete the text for System Calls and Stack Growth*

Swapping

Student TODO: *Complete the text for Swapping*

5.2.2 Interactions Between Components (Student Tasks)

Student TODO: *Complete the text as a team, defining a subsubsection for each interaction, or to note the lack of interactions*

5.2.3 Design Decisions, Motivations and Alternatives

Per Process Quotas and Zero Pages

Student TODO: *Complete the text for Per Process Quotas and Zero Pages*

System Calls and Stack Growth

Student TODO: *Complete the text for System Calls and Stack Growth*

Swapping

Student TODO: *Complete the text for Swapping*

5.3 Implementation Considerations

Per Process Quotas and Zero Pages

Student TODO: *Complete the text for Per Process Quotas and Zero Pages*

System Calls and Stack Growth

Student TODO: *Complete the text for System Calls and Stack Growth*

Swapping

Student TODO: *Complete the text for Swapping*

5.4 Questions that you could be asked

This section must be removed. This is only to give you some hints for your design.

Some questions you have to answer (inspired from the original Pintos design templates), but these are not the only possible questions and we insist that your design should not be based exclusively to answering such questions:

1. Process Quotas
    - When sharing memory between processes the physical frames used should be counted in the quota for each process, how will you achieve this?
2. Zero Pages
    - How will you implement zero pages? How will you support accessing a range of zero memory larger than the swap file? Explain the algorithm required for your implementation.
3. Stack Growth
    - Explain your heuristic for deciding whether a page fault for an invalid virtual address should cause the stack to be extended into the page that faulted.
4. page table management
    - In a few paragraphs, describe your code for locating the frame, if any, that contains the data of a given page.

- How does your code coordinate accessed and dirty bits between kernel and user virtual addresses that alias a single frame, or alternatively how do you avoid the issue?
- When two user processes both need a new frame at the same time, how are races avoided?

5. page replacement and swapping

   - When a frame is required but none is free, some frame must be evicted. Describe your code for choosing a frame to evict.
   - When a process P obtains a frame that was previously used by a process Q, how do you adjust the page table (and any other data structures) to reflect the frame Q no longer has?
   - Explain the basics of your VM synchronization design. In particular, explain how it prevents deadlock. (Refer to the textbook for an explanation of the necessary conditions for deadlock.)
   - A page fault in process P can cause another process Q's frame to be evicted. How do you ensure that Q cannot access or modify the page during the eviction process? How do you avoid a race between P evicting Q's frame and Q faulting the page back in?
   - Suppose a page fault in process P causes a page to be read from the file system or swap. How do you ensure that a second process Q cannot interfere by e.g. attempting to evict the frame while it is still being read in?
   - Explain how you handle access to paged-out pages that occur during system calls. Do you use page faults to bring in pages (as in user programs), or do you have a mechanism for "locking" frames into physical memory, or do you use some other design? How do you gracefully handle attempted accesses to invalid virtual addresses?
   - A single lock for the whole VM system would make synchronization easy, but limit parallelism. On the other hand, using many locks complicates synchronization and raises the possibility for deadlock but allows for high parallelism. Explain where your design falls along this continuum and why you chose to design it this way.

6. Shared Memory

- What data do you need to maintain in order to be able to create and access shared memory regions? What data structure will you be using to maintain the keys?
- After creating a shared memory region, when it is accessed by the second process does the same virtual address need to be returned to the process? Motivate your answer.
- How will you know when to delete the shared memory region? Take this scenario as an example: a process creates a region and 7 other processes access it, i.e. they call *SyscallVirtualAlloc* with the same key, how do you know when you need to delete the key from your list and implicitly the shared memory region? This region should be valid until the last process which 'opened' it 'closes' it, i.e. calls *SyscallVirtualFree*.

Part I

Appendix

Appendix A

The (One) Way to Proceed for Getting a Reasonable Design

A.1 General Considerations

There are multiple strategies to develop a software application, though basically all of them comprise the following four phases:

1. establish the *application requirements* or specification;
2. derive the ways the requirements can be realized / satisfied, i.e. *designing* the application;
3. *implement* the design, e.g. write the corresponding code;
4. check if the implementation satisfies the specification, at least by *testing* (as exhaustive as possible) the developed application.

In practice, a perfect and complete design is not entirely possible from the beginning for most of the projects. So, at least the last three phases actually correspond to a progressive and repeating process, i.e. make a first design, implement it, test the resulting code, see what is working bad or missing functionality, go back and change the design, make the corresponding code changes or additions, test them again and so on.

I want, however, to warn you that even if we cannot make a perfect design from the beginning that does not mean that we do not have to make any design at all and just start writing code. This is a really bad idea. And actually, in my opinion, when you start writing code without a more or less formal design, what you actually have to do is to derive an on-the-fly design. What I mean is that you cannot just write "some" code there, hoping to get

the required functionality. You must think of *how* to code and *what* code to write and this is basically a (hopefully, logical) plan, i.e. a design. Such a strategy, however, results most of the time and for most of the people in just poorly improvisation and requires many returns to the "design" phase to change data structures and code. In short, a lot of lost time and bad results.

Coming back to the idea that we cannot make a complete design from the beginning, there are a few ways to understand this and reasons of having it. Firstly, it is generally difficult to cover all the particular cases, especially for very complex systems and requirements. That means that what you get first is a sort of a general design, establishing the main components of your application and their basic interrelationships. It is not surely that you immediately could start writing code for such a design, but it is very possible to be able to write some prototype, just to see if your ideas and design components could be linked together. On way or another, the next major step is to go deeper for a more detailed design. Secondly, one reason of not getting a complete design from the beginning is just because you want to concentrate on a particular component firstly, and only than to cope with the others. However, this is just a particular case of the first strategy, because it is not possible to deal with one application component without knowing firstly which are the others and how they depend on one another. Thirdly, maybe it is not needed to get a complete detailed design from the beginning, just because the application components are dealt with by different teams or, like in your case, different team members. It is not needed in such a case to deal with the complexity of each application component from the beginning, as each one be will be addressed latter by its allocated team (members), but just try to establish as precise as possible, which are the application components and how they need to interact each other. In your *HAL9000* project the application components are most of the time already established, so what remains for you is only to clarify the interactions and interfaces between them. After such a general design, each team (member) can get independently into a more detailed design of his/her allocated application component. In conclusion, you need to derive at least a general design before starting writing any code and refine that design later.

Take into account, however, that in or project we have distinct deadlines for both design and implementation phases, and that you will be graded for the two relatively independent (thus, design regrading will be done only occasionally). This means that you have to try to derive a very good and as detailed as possible design from the beginning.

Another practical idea regarding the application development phases is that there is no clear separation between those phases and especially between the design and implementation ones. This means that during what we call

the design phase we have to decide on some implementation aspects and, similarly, when we are writing code we still have to take some decisions when more implementation alternatives exists (which could influence some of the application non-functional characteristics, like performance) or some unanticipated problems arise. Even taking into account such realities, in this design document we are mainly interested (and so you have to focus) mainly on the design aspects. But, as I said above, I will not expect you providing a perfect design, which would need no changes during its implementation. However, this does not mean that you are free to come with an unrealistic, incoherent, illogical, hasty, superficial design, which does not deal with all the (clear and obvious) given requirements.

One important thing to keep in mind when you make your design and write your design document is that another team member has to be able to figure out easily what you meant in your design document and implement your design without being forced to take any additional design decisions during its implementation or asking you for clarifications. It is at least your teacher you have to think of when writing your design document, because s/he has to understand what you meant when s/he will be reading your document. Take care that you will be graded for your design document with approximately the same weight as for your implementation.

Beside the fact that we do not require you a perfect design, and correspondingly we do not grade with the maximum value only perfect designs (yet, please, take care and see again above what I mean by an imperfect design), your design document must also not be a formal document. At the minimum it should be clear and logical and complies the given structure, but otherwise you are free to write it any way you feel comfortable. For example, if you think it helps someone better understand what you mean or helps you better explain your ideas, you are free to make informal hand-made figures, schemes, diagrams, make a photo of them or scan them and insert them in your document. Also, when you want to describe an algorithm or a functionality, you are free to describe it any way it is simpler for you, like for instance as a pseudo-code, or as a numbered list of steps. However, what I generally consider a bad idea is to describe an algorithm as an unstructured text. On one hand, this is difficult to follow and, on the other hand, text could generally be given different interpretations, though as I already mentioned your design should be clear and give no way for wrong interpretation, otherwise it is a bad design.

Regarding the fact that design and implementation could not be clearly and completely separated, this is even more complicated in your *HAL9000* project that you start from a given code of an already functional system. In other words you start with an already partially designed system, which

you cannot ignore. This means, on one hand, that you could be restricted in many ways by the existing design and *HAL9000* structure and, on the other hand, that you cannot make your design ignoring the *HAL9000*' code. Even if, theoretically, a design could be abstract enough to support different implementations (consequently, containing no particular code), your *HAL9000* design has to make direct references to some existing data structures and functions, when they are needed for the functionality of the *HAL9000* component your are designing. So, do not let your design be too vague (abstract) in regarding to the functionality directly relying on exiting data structures and functions and let it mention them explicitly. For instance, when you need to keep some information about a thread, you can mention that such information will be stored as additional fields of the "*THREAD*" data structure. Or, when you need to keep a list of some elements, you have to use the lists already provided by *HAL9000* and show that by declaring and defining your list in the way *HAL9000* does, like below:

```
// this is the way HAL9000 declares a generic
   list
LIST_ENTRY myCoolList;

typedef struct _MY_LIST_ELEM {
        ... some fields ...

        // this is needed for linking
           MY_LIST_ELEM in the
        // myCoolList list
        LIST_ENTRY ListElem;
} MY_LIST_ELEM, *PMY_LIST_ELEM;
```

The next sections illustrate the design document structure we require and describe what each section should refer to.

A.2 Application Requirements. Project Specification

A.2.1 "What you have" and "What you have to do"

In this section you have to make clear what you are required to do for each particular assignment of the *HAL9000* project. In the *HAL9000* project, you are already given the assignment requirements, so it is not your job to establish them. You must, however, be sure that you clearly understand them. Having no clear idea about what you have to do, gives you a little

chance to really get it working. Please, do not hesitate to ask as many questions about such aspects on the *HAL9000* forum (on the moodle page of the course) as you need to make all the requirements clear to you. Take care, however, that you have to do this (long enough) before the design deadline, such that to get an answer in time. We will do our best in answering your questions as fast as possible, though we cannot assure you for a certain (maximum) reaction time. You will not excused at all if you say you had not understood some requirements when you will be presenting your design document.

So, for this small section, take a moment and think of and briefly write about:

1. what you are starting from, i.e. what you are given in terms of *HAL9000* existing functionality, and

2. what you are required to do.

A.2.2 "How it would be used"

Making clear the requirements could be helped by figuring some ways the required functionality would be used once implemented. For this you have to describe briefly a few common use-cases, which could later be used as some of the implementation tests.

You could use for this the tests provided with the *HAL9000* code in the "tests/" subdirectory. Take at least a short look at each test case to identify common cases.

A.3 Derive the Application's Design

Generally, you have to follow a top-down design approach, starting with a particular requirement and identifying the inputs it generates to your application (i.e. *HAL9000* OS). Such that you could establish the "entry (starting) points" in your system to start your design from.

Next, you also have to identify the logical objects (i.e. data structures, local and global variables etc.) implied and affected by the analyzed requirement and the operations needed to be performed on them. Also establish if you need to introduce and use additional information (i.e. fields, variables) in order to make such operations possible.

Once you established the information you need to keep in order to dealt with the analyzed requirement, you could decide on the way to keep track of and manage that data. In other words, this is the way you can *identify the*

needed data structures and *operations on them*. There could not necessarily be just one solution. For example, at one moment you could use a linked list or a hash table. In order to decide for one or another, you have to figure out which one helps you more, which one fits better for the most frequent operation and other things like these. Once you decided on the data structures, you have to establish where and how they are (1) *initialized*, (2) *used*, (3) *changed*, and finally (4) *removed*.

This way you could identify the places (e.g. other system components, functions) where you have to add the needed functionality. In terms of *HAL9000* code you could identify the functions needed to be used, changed or even added.

As a result of your requirement analysis you could organize your resulting design like below.

A.3.1 Needed Data Structures and Functions

Describe the *data structures* and *functions* (only their signature and overall functionality) needed by your design and what they are used for. Describe in few words the purpose of new added data structures, fields and functions. As mentioned before, cannot ignore the fact that *HAL9000* is written in C. Thus, describe you data structures in the C syntax referring, when the case, to existing data structures, like in the example given above. If you only need to add new fields in existing data structures, mention only the added fields, not all of them.

```
typedef struct _EXISTENT_DATA_STRUCTURE {
    ...
    int NewField;
    char NewField2;
    ...
} EXISTENT_DATA_STRUCTURE, *PEXISTENT_DATA_STRUCTURE;

typedef struct _NEW_DATA_STRUCTURE {
    ... new fields ...
} NEW_DATA_STRUCTURE, *PNEW_DATA_STRUCTURE;

STATUS NewFunction(void);
```

A.3.2 Detailed Functionality

This should be **the largest and most important section** of you design document. It must describe **explicitly**, in words and pseudo-code the way

your solution works. DO NOT INCLUDE CODE HERE. This is a design document, not a code description one. As much as possible try to describe the design principles, not implementation details.

Give examples, if you think they can make your explanation clearer. You are free to use any other techniques (e.g. use-case diagrams, sequence diagrams etc.) that you think can make your explanation clearer. See Figure A.1 below to see the way images are inserted in a Latex file.

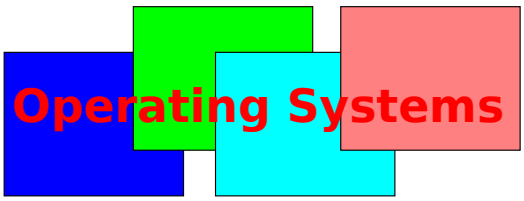


Figure A.1: Sample image

When describing algorithms you are to use or develop, it is very important to also describe them in a formal way, not just as free text description. Commonly, this could be done as a sort of pseudo-code or just as a simple list of logically ordered steps (enumerated list). In your algorithm description you should (i.e. must) refer to the data structures and variables you described in the previous section, following the recommendations in Section **??**. This way you avoid describing your algorithms too vague and reduce the risks of being misunderstood.

Here you have a pseudo-code description of an algorithm taken from http://en.wikibooks.org/wiki/LaTeX. It uses the *algpseudocode* package. Alternatively, you can use any other package and environment of same sort you like.

if $i \geq maxval$ **then**
  $i \leftarrow 0$
else
  if $i + k \leq maxval$ **then**
    $i \leftarrow i + k$
  end if
end if

A.3.3 Explanation of Your Design Decisions

Justify very briefly your design decisions, specifying other possible design alternatives, their advantages and disadvantages and mention the reasons of your choice. For instance, you can say that you decided for a simpler

and easier to implement alternative, just because you had no enough time to invest in a more complex one. Or just because you felt it would be enough for what *HAL9000* tests would check for. This could be viewed as a pragmatical approach and it is not necessarily a bad one, on the contrary, could be very effective in practice.

A.4 Testing Your Implementation

Please note that the *HAL9000* code is provided with a set of tests that are used to check and evaluate your implementation. The *HAL9000* tests could be found in the "tests/" subdirectory, organized in different subdirectories for each different assignments (like, "threads", "userprog" etc.).

To find out the names of all the tests to be run for a module you can build the *RunTests* project and wait for execution to finish.

Actually, this is the first command that will be executed on your implementation when graded, so please, do not hesitate do run it by yourself as many times as needed during your *HAL9000* development, starting from the design phase.

In this section you have to describe briefly each of the given *HAL9000* tests that will be run in order to check the completeness and correctness of your implementation. Take care that your grade is directly dependent on how many tests your implementation will pass, so take time to see if your design take into account all particular usage scenarios generated by all *HAL9000* tests.

A.5 Observations

You can use this section to mention other things not mentioned in the other sections.

You can (realistically and objectively) indicate and evaluate, for instance:

- the most difficult parts of your assignment and the reasons you think they were so;
- the difficulty level of the assignment and if the allocated time was enough or not;
- particular actions or hints you think we should do for or give to students to help them better dealing with the assignments.

You can also take a minute to think what your achieved experience is after finishing your design and try to share that experience with the others.

You can also make suggestions for your teacher, relative to the way s/he can assist more effectively her/his students.

If you have nothing to say here, please remove it.

Appendix B

A Complete and Detailed Design for Module *Threads*

B.1 Assignment Requirement

B.1.1 Initial Functionality

Timer

We are given an implementation of the executive timer (see file `ex_timer.c`), which is based on busy-waiting, i.e. a loop where the time expiration condition is checked continuously, keeping the CPU busy.

Priority Scheduler. Fixed Priority Scheduler

We are given an implementation of a Round-Robin scheduling policy (see function `_ThreadGetReadyThread()` in file `thread.c`), which consider all threads equal, giving them CPUs based on FCFS (First-Come First-Served) principle and only for a predefined time slice. We are also given the implementation of thread switching mechanism (see function `ThreadSwitch` in file `_thread.yasm`).

Priority Scheduler. Priority Donation

Same given code as described at previous section.

Advanced Scheduler (MLFQ). Dynamic-Priority Scheduler

Same given code as described at previous section.

B.1.2 Requirements

Timer

We must change the given executive timer functionality, such that to not use anymore the busy waiting technique in function `ExTimerWait()`, but a blocking mechanism, which suspends the waiting thread (i.e. takes the CPU from it) until the waited timer expires.

Priority Scheduler. Fixed Priority Scheduler

We must implement a priority-based thread scheduling policy (algorithm), which means the scheduler must consider threads' priorities when deciding which thread to be given an available CPU. The main scheduling rule is that when a thread must be chosen, the one with the higher priority must be the choice. The scheduler must be preemptive, which means it must always assure that threads with the highest priorities (considering multiple CPUs) are the ones run at any moment. This could suppose a currently running thread could be suspended, when a new higher-priority thread occurs.

Priority Scheduler. Priority Donation

We must implement (temporary) priority donation as a solution to the "priority inversion" problem. Priority inversion correspond to situations when a thread with a higher priority wait for a thread with a smaller priority (contrary to the priority-based rule, which requires the opposite). Such a situation occur when a thread with a smaller priority succeeds taking a lock, which will be later on required by a higher-priority thread. In order to avoid having the higher-priority thread waiting for smaller-priority threads (others than the lock holder, but having a higher priority than it), the higher thread donates its priority to the lock holder (smaller thread) until the lock will be released. In the meantime, no other "in-between" thread, could block the lock holder and consequently the waiting higher-priority thread.

Advanced Scheduler (MLFQ). Dynamic-Priority Scheduler

We must implement a priority-based scheduler, but differently by the one describe at Section B.1.2, we must establish thread priority dynamically, during thread runtime, based on some given criteria (like time consumed for running, i.e. using, on the CPU, time spent waiting for a CPU or for some other event). The required algorithm is called multi-level feedback queue (MLFQ), suggesting the continuous increase or decrease of thread priorities, based on their runtime behavior.

B.1.3 Basic Use Cases

Timer

A user application could use a timer to have one of its threads periodically (e.g. every one second) increasing a counter and displaying it on the screen. This could be a sort of wall-clock.

Priority Scheduler. Fixed Priority Scheduler

A user application could establish different priorities for its different threads, based on some application specific criteria. Similarly, the OS itself could create a (kernel) thread to handle critical system events, giving that thread a higher priority than those of all other user-application threads.

Priority Scheduler. Priority Donation

This is not directly visible to and controlled by a user application, but has effects on scheduling performance, in the sense that high-priority threads will not be delayed too much when competing for locks with smaller-priority threads (already having those locks). It could be difficult to create a use case (in a user application) to measure the correctness and effectiveness of such a mechanism, but we could image a use case in kernel (as we already have in the given tests). An example in this sense could be the following scenario:

1. start a testing thread, which will take the following steps
2. creates a smaller-priority thread, which takes a lock;
3. after being sure the lock was taken by the smaller-priority thread, creates a second, higher priority thread, which wants to take the same lock; we must see the current lock holder is given (donated) the higher-priority thread;
4. while the smaller-priority thread is still keeping the lock, other threads, with priorities between the small and high threads could be created, which we must see that will not suspend the small-priority thread holding the lock;
5. the small-priority thread releases the lock, and we must see that its priority goes back to its original one;
6. the main thread waits until all created threads terminates, before terminating itself.

Advanced Scheduler (MLFQ). Dynamic-Priority Scheduler

This kind of functionality is also almost invisible from user space and there is no explicit way to control or test it from there. It generally could be tested by observing the way the interactive threads in user applications react to user input. Such threads are blocked most of the time, waiting for use input (e.g. keyboard taste, mouse click), but it is very important to be given a priority boost when awaken, such that to be able to rapidly get the CPU to give the user a fast response. This could only be obtain if MLFQ scheduling is used and the priority changing criteria are well tuned.

B.2 Design Description

B.2.1 Needed Data Structures and Functions

Timer

We will change the `_EX_TIMER` structure in the following way:

```
struct _EX_TIMER
{
    ...

    // keep track of threads waiting (blocked) for the
       timer
    EX_EVENT    TimerEvent;

    // used to place the timer in a global timer list
    LIST_ENTRY  TimerListElem;

    ...
} EX_TIMER, *PEX_TIMER;
```

We add a global list keeping track of all timers in the system and a lock to protect this list while accessed concurrently:

```
struct _GLOBAL_TIMER_LIST
{
    // protect the global timer list
    LOCK                TimerListLock;

    // the list's head
    LIST_ENTRY          TimerListHead;
```

```
};

static struct _GLOBAL_TIMER_LIST m_globalTimerList;
```

Functions that we will change:

- `ExTimerInit()`: add the new timer in the global timer list;
- `ExTimerStop()`: signal waiting threads, the timer is no longer evolving;
- `ExTimerWait()`: replace the busy-waiting with the blocking technique;
- `ExTimerUninit()`: remove the timer from the global timer list.

New functions that we will add to file `ex_timer.c`:

- `ExTimerSystemPreinit(void)`: initialize the global timer list and its associated lock;
- `ExTimerCompareListElems(PLIST_ENTRY t1, PLIST_ENTRY t2, PVOID context)`: compare two timer trigger time in order to keep the global timer list order by timer triggering time;
- `ExTimerCheck(PEX_TIMER timer)`: called on each timer interrupt handling to check for a particular timer if it must be triggered or not;
- `ExTimerCheckAll(void)`: called on each timer interrupt handling to check for all timers in the global timer list if they must be triggered or not.

Priority Scheduler. Fixed Priority Scheduler

We need to take into account the threads' priorities, but as long as this is a field already existing in the `_THREAD` structure, we must only use it.

```
typedef struct _THREAD
    ...

    THREAD_PRIORITY          Priority;

    ...
};
```

The functions (all in `thread.c`) we will make changes in or just use are:

- `ThreadSystemPreinit();`

- `ThreadYield()`: change it such that to consider thread priorities;
- `ThreadUnblock()`: change it such that to consider thread priorities; in particular when a thread with a higher priority of any other running thread is unblock, one of the smaller-priority running threads must be preempted;
- `MutexAcquire()`: keep the mutex's waiting queue ordered by thread priorities, such that when the mutex become available, the thread with the highest priority in the mutex's waiting queue to be unblocked;
- `ExEventWaitForSignal()`: to keep the event's waiting list ordered by thread priorities.
- `ThreadSetPriority()`: force the currently running thread give up the CPU, if its new priority is smaller than that of any thread in the ready list;
- `SmpSendGenericIpi()`: send an inter-processor interrupt (IPI) to require all CPUs recheck the ready list, such that to be sure that the running threads are always the ones with the highest priority; used when a new thread is unblocked.

The new functions that we will add are:

- `ThreadComparePriorityReadyList(PLIST_ENTRY e1, PLIST_ENTRY e2, PVOID Context)`: compares the two threads, `t1` and `t2`, based on their priority; it will be used to keep the thread lists ordered based on their priorities (descending order);
- `ThreadYieldForIpi`: to be run by CPUs while receiving an IPI, when a running thread must be preempted by an unblocked thread with a greater priority.

Priority Scheduler. Priority Donation

Priority donation supposes giving a thread a new temporal priority, while that thread is holding a mutex. We must be able to restore such a thread's priority back to its original one, so logically, we must keep track of both types of priorities. The two priorities are called in related literature (or other real OSes) the *real priority* (or original priority), the priority established at thread creation or changed during thread execution by the thread itself, and the *effective priority* (or actual priority), the priority dynamically established by

the OS (based on different internal criteria) and considered by the priority-based OS' scheduler. Based on this basic deduction and on the analysis described in Section B.2.3, we established the need for the following new fields in existing data structure or new data structures. While there were already a field "Priority" associated to each thread, and some tests consider it as the effective one, we let its name unchanged, and only added a new field for what we consider to be the real priority.

In file thread_internal.h:

```
struct _THREAD
{
    LOCK                PriorityProtectionLock;

    _Guarded_by_(PriorityProtectionLock)
    THREAD_PRIORITY     Priority;        // the effective
       priority (already defined)

    ...

    // Used for priority donation
    _Guarded_by_(PriorityProtectionLock)
    THREAD_PRIORITY     RealPriority;    // the real (
       original) priority

    LIST_ENTRY          AcquiredMutexesList; // the list
       of mutexes held by thread
    PMUTEX              WaitedMutex;     // the mutex
       thread waits for

    ...
}
```

In file mutex.h:

```
struct _MUTEX
    ...
    LIST_ENTRY      AcquiredMutexListElem; // elem in
       list of mutexes acquired by a thread
    ...
} MUTEX, *PMUTEX;
```

Functions that must be changed (in mutex.c and thread.c) are:

- MutexAcquire(): donate priority of blocking thread (in case the mutex in already acquired), if its priority is higher than that of mutex holder;

- `MutexRelease()`: recompute the priority of the thread releasing the mutex;
- `ThreadSetPriority()`: take into account both real priority, which is changed by the functions, and the current effective (donated) one;
- `ThreadGetPriority()`: assure the actual (i.e. effective, donated, if the case) priority is the one returned;

New functions that we will implement (in `thread.c`) are:

- `ThreadDonatePriority()`: called in `MutexAcquire()` to donate priority to a mutex holder and also deal with the nested donation aspect;
- `ThreadRecomputePriority()`: called in `MutexRelease()` to recompute the priority of a thread just releasing a mutex, taking into account its real priority, but also priorities donated due to other mutexes that thread still holds.

Advanced Scheduler (MLFQ). Dynamic-Priority Scheduler

To be determined by students. Read the given documentation, while all the needed details are described there.

B.2.2 Interfaces Between Components

To be determined by students.

B.2.3 Analysis and Detailed Functionality

Timer

Replacing the busy-waiting requires a mechanism to block a thread until the waited event (i.e. a particular time moment) occurs. We immediately noted the function `ThreadBlock()` (in file `thread.c`) provides such a functionality. We took a look of places `ThreadBlock()` was already used, to see the way the blocking mechanism is used. One good example was in function `MutexAcquire()`, where we noted that before blocking the current thread (i.e. the one waiting for the mutex), it was inserted in a waiting list, associated to that mutex. We also noted that the blocked thread was unblocked in function `MutexRelease()`, by calling the function `ThreadUnblock()` on the blocked thread removed from the waiting list. So, our first idea was to create a similar list for each timer instance, where to keep threads waiting for that timer to

expire, using ThreadBlock() function to block a thread and ThreadUnblock() to unblock it.

However, we noted that the ThreadBlock() function was called in function ExEventWaitForSignal() also. Investigating a little further, we noted that the executive event was a general wait to manage threads waiting for a particular (not specified, so general) event, by blocking the waiting threads and unblocking them when the event occurred (i.e. signaled). This seemed to be a higher level interface to the block / unblock mechanism, so we decided to use it.

As a result, we decided to associate an executive event to each timer. This translates in an additional EX_EVENT field in the _EX_TIMER structure, like illustrated in Section B.2.1.

Once we had established the TimerEvent field, we had to determine three aspects related its usage:

1. when and how to initialize it;
2. where and how to use it to block a thread waiting for the timer to expire;
3. where and how to use it to unblock the waiting threads, when the timer expires.

The decisions we took in these sense were the following ones:

1. logically, we could (and must) initialize a timer's event, when that timer is initialized itself, i.e. in the function ExTimerInit(); we could do it by calling the ExEventInit() function, with the event type "ExEventTypeNotification" and not signaled initially.
2. replacing the busy-waited loop and blocking the waiting thread could be done very simple in function ExTimerWait(), by simply calling the function ExEventWaitForSignal() on the timer's event;
3. unblocking the thread should be done very simple by calling the ExEventSignal() function of the timer's event, though where to do this was not pretty obvious; in order to determine this, we looked in more details at the condition checked by the initial code in the busy waiting loop in function ExTimerWait() and noted there were actually two conditions:
   (a) one (the second, actually) that checked in the timer was still started (checking the Timer->TimerStarted field); this immediately lead us to the conclusion that one place to call ExEventSignal() was the function ExTimerStop(), where the TimerStarted field was set to FALSE;

(b) another one that checked if the system time (returned by function `IomuGetSystemTimeUs()`) had reached a value equal to or bigger than the timer's triggering time; this suggested us that the other place the `ExEventSignal()` should be called was where the time passage could be observed; investigating in this direction, we found out that this was related to the timer interrupt occurrence and also that the function `ExSystemTimerTick()` (in file `ex_system.c`) was called every time a timer interrupt occurred; so, we decided to have a function `ExTimerCheck()` to be called from `ExSystemTimerTick()`; the way that function works is described below.

The function `ExTimerCheck()` must check if the system's time is bigger than or equal to a timer's triggering time and if the case, call the `ExEventSignal()` on that timer's event. This looks like this:

```
if (system_time >= timer_triggering_time)
    call ExEventSignal() on timer event field
```

We also noted that the same checking must be done on all possible timers existing in the system, so we needed a way to keep track of all of them and to call their `ExTimerCheck()` function. This is why we decided to keep a global list with all the timers in the system. We also decided to have a lock protecting that list for concurrent accesses (it would be possible for multiple threads running on different CPUs to concurrently create or remove different timers). We placed the list and its lock in a structure called `struct _GLOBAL_TIMER_LIST` and created a global variable of that type, called `m_globalTimerList`.

As with any other global variable or data structure field we had to determine where to initialize it and its lock, where to use it (i.e. in our case add new timers or remove them) and where to destroy it. We took the following decisions in this context:

1. create a new function `ExTimerSystemPreinit()` to be called where other functions initializing OS's components are called, i.e. in function `SystemPreinit()` (file `system.c`); this function would simply initialize the global list of timers (by calling `InitializeListHead()`) and the corresponding lock (by calling `LockInit()`);

2. adding a new timer in the list will be done in function `ExTimerInit()`, by performing the following steps:

   (a) acquire the lock that protects the global list of timers (calling `LockAcquire()`);

(b) insert the timer's structure in the global list of timers (by calling `InsertOrderedList()`, for reason explained below);

(c) release the lock that protects the global list of timers (calling `LockRelease()`).

3. removing a timer from the global list will be done in function `ExTimerUninit()`, by performing the following steps:

   (a) acquire the lock that protects the global list of timers (calling `LockAcquire()`);

   (b) remove the timer's structure in the global list of timers (calling `RemoveEntryList()`);

   (c) release the lock that protects the global list of timers (calling `LockRelease()`).

4. iterate the global list when a timer interrupt occurs (i.e. in function `ExSystemTimerTick()`) and call the `ExTimerCheck()` for each thread in the global list; actually, in order to keep the current coding style, we will declare the global list of timers as a static variable (so visible only in file `ex_timer.c`) and a new function `ExTimerCheckAll()` (declared `ex_timer.h` and defined in `ex_timer.c`) to be called from `ExSystemTimerTick()` and performed the mentioned steps.

   (a) acquire the lock that protects the global list of timers (calling `LockAcquire()`);

   (b) iterate the global list of timers (e.g. by calling `ForEachElementExecute()` or using any other way to iterate a list) and call the `ExTimerCheck()` for each element in that list;

   (c) release the lock of the global list of timers (calling `LockRelease()`).

We discover an optimization we could apply when checking for timers' triggering time, in order to shorted the time spent handling the timer interrupt. We noted that only timers having their trigger time smaller than or equal to the system time must be signaled, so we decided to keep the global list of timers ordered (ascending) by timers' trigger time. This way, once a timer with its trigger time bigger than the system's time is encountered, the list iteration could be stopped. This is why we will call `InsertOrderedList()` in `ExTimerInit`. That function expects as a parameter a function that know to compare two timers, which we decide to be a new function `ExTimerCompareListElems()`. This function works as follows:

1. obtains from its first two parameters, which are of the generic type PLIST_ENTRY, two pointers to EX_TIMER structure, using the CONTAINING_RECORD macro;

2. compare the two timers by calling the function ExTimerCompareTimers().

Take care that using the ForEachElementExecute() function to iterate the global list of timers could traverse the entire list, even if that list is ordered, so making your optiomization ineffective. Though, we could change this behaviour setting correspondigly the forth parameter (AlMustSucceed) of ForEachElementExecute() function and by carrefully implementing the ExTimerCompareTimer() function. Take a look at the implementation of function ForEachElementExecute() to see exactly how its forth parameter must be specify and how the ExTimerCompareTimer() function must be implemented.

Priority Scheduler. Fixed Priority Scheduler

The priority-based scheduler's policy requires that at any moment the highest priority threads (from the ones wanting for the CPU, i.e. the so-called ready threads) to be running. This rule implies two requirements we must deal:

1. when there are more threads we must choose from, the one with the highest priority must be chosen;

2. an unblocked or a newly created thread with a higher priority than any other running thread must preempt a smaller priority thread and be given the released CPU.

In order to satisfy the first requirement, we decided to order all thread lists in the system, based on their priorities, in the descendant order, such that to always have the thread with the highest priority in front of the list. We had identified all such lists, which are:

1. the ready list (see field ReadyThreadsList of structure _THREAD_SYSTEM_DATA, file thread.c);

2. mutexes' lists (see field WaitingList of structure _MUTEX, file mutex.h);

3. events' lists (see field WaitingList of structure _EX_EVENT, file ex_event.h).

For keeping the thread lists ordered, we will replace the usage of InsertTailList() with the function InsertOrderedList(), which will be given as parameter a thread comparison function ThreadComparePriorityReadyList() (described below), in the following functions:

- in `ThreadUnblock()` for inserting the unblocked thread in `ReadyThreadsList` ordered descending by priority;
- in `ThreadYield()` for inserting the yielding thread (i.e. the one giving up the CPU) in `ReadyThreadsList` ordered descending by priority;
- in `MutexAcquire()` for inserting the blocking thread (i.e. the one waiting for the mutex) in `WaitingList` ordered descending by priority;
- in `ExEventWaitForSignal()` for inserting the blocking thread (i.e. the one waiting for the event occurrence) in `WaitingList` ordered descending by priority.

The `ThreadComparePriorityReadyList()` function works the following way:

- obtains from its first parameter `e1`, which is of the generic type `PLIST_ENTRY`, a pointer to a `_THREAD` structure, using the `CONTAINING_RECORD` macro, whose the first parameter is the list element, the second is the `THREAD` structure containing that list element, and the third is the name of the list element field in the THREAD structure, which is the "`ReadyList`":

```
PTHREAD pTh1;
pTh1 = CONTAINING_RECORD(e1, THREAD, ReadyList);
```

- similarly, obtains from its second parameter `e2` a pointer to a `_THREAD` structure:

```
PTHREAD pTh2;
pTh1 = CONTAINING_RECORD(e2, THREAD, ReadyList);
```

- compare the two threads' priorities and return the result such that to order the list in a descendant way (i.e. negative, if second thread's priority is less the that of the first, positive if the opposite, and zero if equal):

```
prio2 = ThreadGetPriority(pTh2);
prio1 = ThreadGetPriority(pTh1);

compare_and_return_result(prio1, prio2);
```

Having the ready list ordered by thread priority implied no need to change the `_ThreadGetReadyThread()` function, which chooses a thread from ready list to be given an available CPU, while choosing the first thread in ready list (by calling `RemoveHeadList()`) would return the thread with the highest priority, exactly what the priority-based policy requires.

Another aspect we should take care about when scheduling threads based on their priorities (besides the rule of always choosing the thread with the highest priority from a thread list) is the case more threads have the same priority, in particular, the same highest priority. In such a case, a fair scheduler would give equal chances to all such threads. This could be provided by using the Round-Robin (RR) policy (the one already being implemented in *HAL9000*), which would take the threads in the order they were added to the ready list and will give them CPUs for an established amount of time (time quantum or slice), placing them back in ready list when their allocated time slice expires. The RR strategy must manage the ready list in a FIFO manner, i.e. appending at the end of the list a thread suspended due to its time quantum expiration. This could be managed on our priority-ordered ready list, if the `InsertOrderedList()` function would insert a thread with a particular priority after all threads with the same priority already in ready list (i.e. at the end of its priority class), while doing so it would be the equivalent of appending to a list containing only threads of that priority. We looked at the implementation of `InsertOrderedList()` and noted that it complied these requirements based on the given comparison function (see above), which means that our scheduler would handle threads with the same priority based on the RR policy.

In order to satisfy the preemption requirement, we searched for situations (and corresponding functions) when a thread becomes a new competitor for CPUs, besides the existing one. The threads competing for CPUs are the running threads and the threads in the ready list, having the running threads with a higher priority than all those in ready list. However, a newly arrived thread could have a higher priority than some of those running at that moment and this is why our scheduler should be called to preempt one of the running threads if having a smaller priority than the newly arrived one, or inserting the new thread in the ready list, in the opposite case.

We identified such situation for:

1. a newly created thread (see function `ThreadCreate()`, which further calls `ThreadCreateEx()`);

2. an unblocked thread (see function `ThreadUnblock()`).

However, when we looked in more details at function `ThreadCreateEx()`, we noted that a newly create thread is set active (i.e. ready) by calling the `ThreadUnblock()` function, which simply inserts that thread in the ready list. This is similar to unblocking any other thread when the condition it was waiting for is fulfilled (e.g. a mutex is released, an event is signaled — see functions `MutexRelease()` and `ExEventSignal()`, where `ThreadUnblock()` is

called). So, we concluded that `ThreadUnblock()` function is the only place we should impose the preemption functionality of our priority-based scheduler, based on the following additional logic:

Require: `unblocked_thread`, `ready_list`, `min_priority_running_threads`
Ensure: Preempt a running thread (on any CPU), if smaller than the unblocked one
new_prio = ThreadGetPriority(unblocked_thread)
min_prio = GetMinPriorityOfRunningThreads
InsertOrderedList(ready_list, unblocked_thread)
if *new_prio* > *min_prio* **then**
Preemption() ▷ Preempt one smaller-priority running thread
end if

A CPU preemption could be implemented by using the `SmpSendGenericIpi()` function to send an interrupt (i.e. IPI) to all CPUs. This way the CPUs will interrupt their execution and switch to other threads in the ready list, if they have a higher priority than the currently running threads.

Sending an IPI to all CPUs could be done the following way:

```
SMP_DESTINATION dest = { 0 };

SmpSendGenericIpiEx(ThreadYieldForIpi, NULL, NULL, NULL,
    FALSE, SmpIpiSendToAllIncludingSelf, dest);
```

The function `ThreadYieldForIpi()` given as the first argument would be executed by each CPU when handling the received IPI and should check the condition mentioned before, taking required actions. We could immediately note that the function `ThreadYield()` provides exactly this kind of functionality. Besides, it protects the ready list with a lock while investigating it, a condition which is really needed to avoid multiple CPUs taking the same threads for being run.

Yet, we have a problem with calling `ThreadYield()` when handling an IPI. `ThreadYield()` switches the CPU from current thread to another one, but it is not recommended to do something like this when an interrupt is handled. Doing so, the interrupt remains only partially handled and this could affect the next interrupts (remaining masked or being delayed indefinitely). Though, there is a solution to this problem, which is used in `ThreadTick()`. Function `ThreadTick()` is called from `ExSystemTimerTick()` when handling the timer interrupt and has the role of increasing different time-related counters and checking if the currently running thread has not yet consumed entirely its allocated CPU time. If it is the case, the current thread must be suspended and CPU switched to another thread. Here is where the functionality of `ThreadYield()` is also needed. But, instead of directly

calling ThreadYield(), we can note that it is only the flag pCpu->ThreadData.YieldOnInterruptReturn that is set to TRUE. Doing that, has the same effect, i.e. calling ThreadYield(), but only at the appropriate moment, after the interrupt handling is finished, just before returning to the interrupted thread. This happens in function _IsrInterruptHandler(), where the YieldOnInterruptReturn flag is checked (by calling ThreadYieldOnInterrupt()) and if set, ThreadYield() is called. In conclusion, instead of directly calling ThreadYield() in our ThreadYieldForIpi() function, we should set the flag YieldOnInterruptReturn of the current CPU.

Another aspect we should manage is the way to find out if preemption is needed or not when a thread is unblocked, even in case of concurrent CPUs. This means that while comparing the unblocked thread's priority to those of the currently running threads, we must be sure the investigated CPUs will not be switched to other threads. We found out two ways to manage correctly this concurrency:

1. always send an IPI to all CPUs, letting them call ThreadYield(), whose concurrent execution is synchronized by a lock (as mentioned before); this is the simplest way, though not the most efficient one, as all CPUs will be interrupted, even if the unblocked thread is smaller than all currently running ones, so must not preempt any one of them;

2. keep a protected list of all running threads or just a global variable having the value of the minimum priority of all running threads; the access to that variable must also be protected by a lock.

We chose the first alternative, being more simple, which means we have no checking anymore in the algorithm of ThreadUnblock() described above, but just trigger the preemption mechanism by calling SmpSendGenericIpi().

After further investigations, we found out that even if our priority-based scheduler does not change the priority of threads, letting them as established at thread creation (supposed to be controlled by user applications those threads belong to), there is still another place, where a thread priority could be changed. This is in function ThreadSetPriority(), which could be called only by a running thread to change its own priority. In such a case, the scheduler must also be called to take into account the changed priority. We identified two situations:

1. if the thread calling ThreadSetPriority() increased its priority, this would not change the current scheduling configuration, while it is supposes that the thread's previous priority had already been higher than those of all threads in ready list, so increasing it even more, produces no effects;

2. if the thread calling `ThreadSetPriority()` decreased its priority, there could be two subcases:

   (a) if the new priority is higher than those of all threads in the ready list, noting should happen, while this is equivalent to the previous case;

   (b) if the new priority is smaller than that of any thread in the ready list, then the currently running thread must give up the CPU in favor of a higher-priority thread in the ready list; this could be done very simple by calling `ThreadYield()` function.

Priority Scheduler. Priority Donation

As mentioned in Section B.2.1, we firstly established the need for two types of priorities associated to a thread: real and effective.

Ignoring the priority donation problem, the real priority is the one established at thread creation (in function `ThreadCreateEx()`) or when the thread itself changes its priority (in function `ThreadSetPriority()`). Correspondingly, we assign values to the real priority field `RealPriority` in function `_ThreadInit()` (called by `ThreadCreateEx`) and in function `ThreadSetPriority()`. Those values correspond to the functions' priority parameter and, if no priority donation takes place for a thread, are equal to the effective priority field `Priority`.

Though, while we must solve the *priority inversion* problem based on priority donation, we could have at one moment different values for the two priorities and must handle differently them. The priority the scheduler must consider when taking its decisions must be the effective priority, i.e. the donated priority, if a priority is donated to it (logically, does not make sense to donate a higher priority to a thread, if that priority is not considered by scheduler).

The priority inversion problem is usually defined (see the lecture slides related to priority-based scheduling) in relation to mutexes. The main idea is that while holding a mutex, a lower-priority thread could be suspended (due to priority-based policy) by higher priority threads. This is not a problem until such a higher-priority thread also wants to take the mutex. Being already acquired, the higher-priority thread must wait, being blocked (so, ceasing the CPU) and giving the mutex holder a change to run again. However, if other threads with priorities between the two mentioned before are running, they keep from running both the lower thread (mutex holder) and, indirectly, the higher thread waiting for the mutex. This looks like the priority rule was switched, a higher one waiting for a smaller one, like if there

priorities would have been switched (this is why the problem is called priority inversion). This could lead to critical problems, when critical high-priority threads could be delayed indefinitely by smaller priority threads, so a good OS (like ours) tries to solve it. One solution to this problem is the priority (temporal) donation technique, which consists in the blocking high-priority thread donating its priority the the smaller-priority mutex holder, in order to boost it such that no in-between thread be able to delay the high one. Obviously, the donated priority must be kept only while the mutex holder has the mutex. When the mutex is released the donated priority must be lost and the thread should be restored to its real one. If this were the only possible case, the implementation of priority donation solution would be very simple. This is not the case however, as we will see below. However, it is clear from the problem definition that we have to handle the two priorities of a thread in the following functions:

1. `MutexAcquire()`, where priority donation could happen, and
2. `MutexRelease()`, where priority restoration must happen, if needed.

Basically, in `MutexAcquire()` we should do the following on the execution path corresponding to the mutex being held, when the current thread must be blocked:

```
while (Mutex->Holder != pCurrentThread)
{
    crtThPrio = ThreadGetPriority(pCurrentThread);
    holderPrio = ThreadGetPriority(Mutex->Holder);

    if (crtPrio > HolderPrio)
    {
        // priority donation
        Mutex->Holder->Priority = crtThPrio;
    }
}
```

Similarly, in `MutexRelease()` we must restore the priority of mutex holder to its original value:

```
if (Mutex->Holder->Priority != Mutex->Holder->
   RealPriority)
{
    Mutex->Holder->Priority = Mutex->Holder->RealPriority
        ;
}
```

In practice it is not so simple, though, because a thread could hold simultaneously multiple mutexes. In that case it could be donated multiple priorities (so, its effective priority being boosted multiple times), due to the multiple mutexes it held. In such a case, it would not be right to restore that thread's priority to its real one when releasing one of its acquired mutexes, as it could still held other ones. The correct solution should take into account both the thread's real priority and the ones donated due the mutexes still being held by that thread. The formula would be something like: $max(real_priority, max(donated_priorities))$

The reason the real priority should be considered is that the thread could change its real priority to a higher value, after being donated a priority due to a mutex is held.

There are different strategies to keep track of donated priorities, but the one we considered is to maintain for each thread a list of the mutexes it holds. Noting that each mutex is associated a waiting list, i.e. a list of all thread waiting for that mutex, it is very simple to see which such threads have a greater priority than the mutex holder, and consider such a priority as a donation. This is the reason we defined the field `AcquiredMutexesList` for each thread. As with any variable, let us see how it is initialized and used:

1. `AcquiredMutexesList` initialization should be done at thread creation and we chose to do it in `_ThreadInit`;

2. inserting an element (i.e. a mutex) in that list must be done when a mutex is acquired by the thread, i.e in function `MutexAcquire()` on the execution path corresponding the the mutex being acquired (after the while loop, waiting for the mutex to become available);

3. removing an element (i.e. a mutex) from that list must be done when a mutex is no longer held by a thread, so in function `MutexRelease()`.

Because we must add `MUTEX` structures to the `AcquiredMutexesList`, we need to add a `LIST_ENTRY` field in that structures, which we name `AcquiredMutexListElem`. Using this field, steps 2 and 3 described above could be done like this: `InsertTailList(GetCurrentThread()->AcquiredMutexList, &Mutex->AcuiredMutexListElem);` for the former and `RemoveEntryList(&Mutex->AcuiredMutexListElem)` for the later. Take a look of the implementation of `InsertTailList()` and `RemoveEntryList()` functions to understand their functionality and required parameters.

The next logical step we must establish is the more complex way a thread's priority is recomputed, when that thread releases one of its held mutexes. So the very simple algorithm described above in function `MutexRelease()` was replaced by another one that we will place in function `ThreadRecomputePriority()`, which basically performs the following steps:

1. initialize a current maximum value to the thread's real priority;
2. iterate the `AcquiredMutexesList` of the thread;
3. for each mutex, iterate its waiting list;
4. for each waiting thread in that list, compares its effective priority to the current maximum, and if bigger, updates the maximum to the new value;
5. set the thread's effective priority `Priority` to the maximum value found.

We will always use the `ThreadGetPriority()` function to correctly get the thread's effective priority. Take care that while iterating a mutex's waiting list in Step 3 above, that list could be concurrently changed by a thread waiting for the corresponding mutex, so entering the waiting list of that mutex. In ordor to avoid race conditions, we must take the lock of the mutex while iterating through its waiting list.

Another problem described in relation to priority inversion is the case a thread holding a mutex is waited by a higher priority thread (which, as established, donates its priority to the mutex holder), but the waiting thread is in its turn the holder of another mutex, which could be later on waited by an even higher-priority thread. The latter thread, would donate its priority to the second we mentioned, but if this donation would not go even further to the first (smallest) thread, the donation would not be effective. This is what lead to the nested priority requirements. However, in order to be able to manage it, we need to know for each thread, when being donated a priority (due to holding a mutex), if that thread is not waiting in its turn after another thread (holding another mutex, in its turn). In order to manage this, we decided to add a new field `WaitedMutex` in the `THREAD` structure, to keep a pointer to a mutex that is waited for by the thread. In case there is no such waited mutex, the pointer would be NULL. We must note that there is no need, in this case for a list of waited mutexes, while, the thread being a single, sequential execution, there is no way to wait for multiple mutexes in the same time (i.e. while being block waiting for a mutex, no execution of that thread take place, such that to be able to attempt taking another mutex).

The initialization to NULL of the field `WaitedMutex` will be done in `_ThreadInit()`, while a valid value (i.e. a pointer to a real mutex) will be done when a thread is going to be blocked, waiting for an already acquired mutex, which is in function `MutexAcquire()`, just before calling `ThreadBlock()`.

The usage of the `WaitedMutex` must be done in `MutexAcquire()` also, when a thread is going to be blocked, but in relation to the priority donation for

the mutex's holder. We want to place the priority donation algorithm in a function called `ThreadDonatePriority()`, which will perform the following steps:

1. compare the donor's priority with the mutex holder's one, and if the former's in greater, performs the donation, i.e. assigns the mutex holder's effective priority to that of the donor's effective priority;

```
donor = GetCurrentThread();
if (ThreadGetPriority(donor) > ThreadGetPriority(
   MutexHolder))
{
  MutexHolder->Priority = donor->Priority;
}
```

2. if the mutex holder is in its turn waiting for another mutex, the donation must go further, like

```
MutexHolder->WaitedMutex->Holder->Priority =
   MutexHolder->Priority;
```

3. the same nested mechanism must be repeated until the end of the waiting chain, i.e. until a thread waiting for no mutex.

Finally, when a thread succeeds acquiring a mutex it waited for previosly, its `WaitedMutex` must be set to NULL again. This must be done on the execution path corresponding the the mutex being acquired (after the while loop, waiting for the mutex to become available).

Another problem we have to deal with in relation to priority inversion is about a thread changing its own real priority, in function `ThreadSetPriority()`. It is possible for a thread to increase its priority to a value higher than its currently donated priority (i.e. its effective priority), in which case, obviously, the effective priority must be given the same value like the current real one. A very simple way (though, not so efficient) to compute the thread's new effective priority would be to call the function `ThreadRecomputePriority()`.

Looking at our strategy to deal with priority donation, we could note that our decision to change at one moment a mutex holder's effective priority could interfere with another decision (and strategy) taken by the priority-based scheduler, namely to keep thread lists ordered based on threads' effective priorities. This is because the thread whose effective priority is changed could be in an ordered thread list and our change could break that order, making later scheduler's decisions incorrect. So, we should consider keeping ordered the list a thread is in when its priority is changed. This could be

easily done by reordering such a list after changing the priority of the thread, e.g. by removing the thread from the list and inserting it back in its new position.

One challenge we face is to know the list a thread is in. A thread could be running, if its state is `ThreadStateRunning`, in the ready list, if its state is `ThreadStateReady` or in a waiting list, if its state is `ThreadStateBlocked`. In the first case, the thread is in no list. In the second, it is in the unique global ready list. In the third case, however, we cannot know by default which waiting list a thread is in, because there could be many such lists (e.g. each mutex / event has its own waiting list) and the current code keeps no information about this. Yet, it would not be difficult to add such a field to the `THREAD` structure and update it correspondingly when a thread is inserted in a waiting list or removed from it.

If the donated thread is running, we must do nothing else, while increasing the priority of an already running thread does not change its position relative to the threads in ready list, which had a smaller priority even before.

While changing the effective priority of a thread during the (nested) priority donation process, we can face race conditions, because there could be multiple donations taking place in the same time: for instance, multiple threads trying to get different mutexes, whose holders are all waiting for the same mutex, so there could be concurrent attempts to change the priority of that mutex holder. Also, if that thread wants to change its own priority, calling `ThreadSetPriority()` function, there is an additional competing attempt to change that thread's effective priority. Because of race conditions like these, a thread's priority fields must be protected by a lock. We add for this the `PriorityProtectionLock` lock.

Advanced Scheduler (MLFQ). Dynamic-Priority Scheduler

To be determined by students (for teams of four members). Read the given documentation, while all the needed details are described there.

B.2.4 Explanation of Your Design Decisions

Timer

To be determined by students, if they have other alternatives or note some inconsistencies in the given design.

Priority Scheduler. Fixed Priority Scheduler

To be determined by students, if they have other alternatives or note some inconsistencies in the given design.

Priority Scheduler. Priority Donation

To be determined by students, if they have other alternatives or note some inconsistencies in the given design.

Advanced Scheduler (MLFQ). Dynamic-Priority Scheduler

To be determined by students, if they have other alternatives or note some inconsistencies in the given design.

B.3 Tests

Your are given in your *HAL9000* the tests your solution will be checked against and evaluated and you are not required to develop any addition test. Though, even if the tests are known, it would be helpful for you during the design phase to take a look at the given tests, because that way you can check if your design covers all of them. It would be sufficient for most of tests to just read the comments describing them.

In this section you have to list the tests affecting your design and given a short description of each one (using you own words).

Timer

To be determined by students.

Priority Scheduler. Fixed Priority Scheduler

To be determined by students.

Priority Scheduler. Priority Donation

To be determined by students.

Advanced Scheduler (MLFQ). Dynamic-Priority Scheduler

To be determined by students.

B.4 Observations

It was an interesting subject to work on. It required much time than I estimated. I learned that a good design is not a trivial thing to do and requires some time to be done.

B.5 Questions that you could be asked

This section must be removed. This is only to give you some hints for your design.

Some questions you have to answer (inspired from the original Pintos design templates), but these are not the only possible questions and we insist that your design should not be based exclusively to answering such questions:

1. timer
    - Briefly describe what happens in a call to *ExTimerWait()*, including the effects of the timer interrupt handler.
    - What steps are taken to minimize the amount of time spent in the timer interrupt handler?
    - How are race conditions avoided when multiple threads call *ExTimerWait()* simultaneously?
    - How are race conditions avoided when a timer interrupt occurs during a call to *ExTimerWait()*?
2. priority scheduler
    - How do you ensure that the highest priority thread waiting for a mutex or executive event wakes up first?
    - Describe the sequence of events when a call to *MutexAcquire()* causes a priority donation. How is nested donation handled?
    - Describe the sequence of events when *MutexRelease()* is called on a lock that a higher-priority thread is waiting for.
    - Describe a potential race in *ThreadSetPriority()* and explain how your implementation avoids it. Can you use a lock to avoid this race?
3. advanced scheduler (MLFQ)

Table B.1: MLFQ Tracing Example

timer ticks	recent_cpu			priority			thread to run
	A	B	C	A	B	C	
0							
4							
8							
12							
16							
20							
24							
28							
32							
36							

- Suppose threads A, B, and C have nice values 0, 1, and 2. Each has a *recent_cpu* value of 0. Fill in the Table B.1 (note: you can use `http://www.tablesgenerator.com/` to easily generate Latex tables) showing the scheduling decision and the priority and *recent_cpu* values for each thread after each given number of timer ticks:
- Did any ambiguities in the scheduler specification make values in the table uncertain? If so, what rule did you use to resolve them? Does this match the behavior of your scheduler?
- How is the way you divided the cost of scheduling between code inside and outside interrupt context likely to affect performance?